

REX & PARCOURS TERRAIN

Tools, agentique et RAG sous Open WebUI

Comment rationaliser les usages en
structurant les workflows et réduisant les
appels LLM ?

Meetup Lorient
Yoann Sola & Xavier Daull



Problème: une conversation trop libre avec un chatbot IA coûte cher + sortie ni garantie ni maîtrisée

Irritants observables

- Les utilisateurs réécrivent le même prompt de 30 lignes
- Les documents sont injectés sans contexte
- Tâche complexe: novice = poule avec un couteau, expert = peut consommer beaucoup de tokens
- La synthèse ou les autres outputs sont freestyle bien que impressionnants à priori

Décision de conception

Chaque élément récurrent devient une brique nommée: template, skill, KB ou tool.

Le prompt libre reste possible, mais il n'est plus le chemin principal. Il peut aussi être analysé pour en déduire des patterns (besoins, processus, output)

Rationaliser: réduire la variabilité inutile

On ne rationalise pas seulement les coûts; on rationalise les chemins de réponse.

Coûts d'inférence

Éviter d'injecter trop de contexte; appeler les connaissances seulement quand c'est utile.



Vitesse de dev

Créer des briques simples: prompts, tools, modèles presets, KB, connecteurs.



Appropriation métier

Structurer la saisie et donner une UX familière pour les utilisateurs non-IA.



Tout cela dans un environnement on premise et air gapé !

Workflow utilisateur “par défaut”

1

Choisir un preset

métier, droits
Choix du package :
modèle + KB + tool

2

Lancer un template

slash command +
formulaire

3

Exécuter le tool

analyse, synthèse,
génération

4

Valider le livrable

traces, incertitudes,
reprise

La valeur vient moins du “meilleur modèle” que de la structuration du choix laissés à l'utilisateur au bon moment et à la variabilité de l'IA générative.

Open WebUI: un socle technique, pas juste une UI



Fournir une UX familière, tout en exposant des briques industrialisables: modèles, prompts, connaissances, skills, tools et fonctions.

- ChatGPT-like pour l'adoption
- Slash prompts et formulaires pour cadrer la saisie
- Knowledge Bases et recherche pour relier la conversation aux corpus
- Tool Calling natif pour externaliser les étapes déterministes

Moins d'appels

Saisie structurée

UX familière

Découplage

Extensible

Point clé

OWUI permet de commencer simple, puis d'externaliser la connaissance ou l'orchestration sans changer l'expérience utilisateur.

À éviter

Faire porter au prompt système ce qui devrait être une option utilisateur, un skill, une KB ou un tool.

La boîte à outils Open WebUI

Prompt template

“contrat” de saisie

Skill

instructions réutilisables,
juste au besoin

Tool

script / calcul / API

Action

Ajout de boutons sous la
réponse du LLM

Knowledge Base

corpus + recherche

Models

assemblage contrôlé

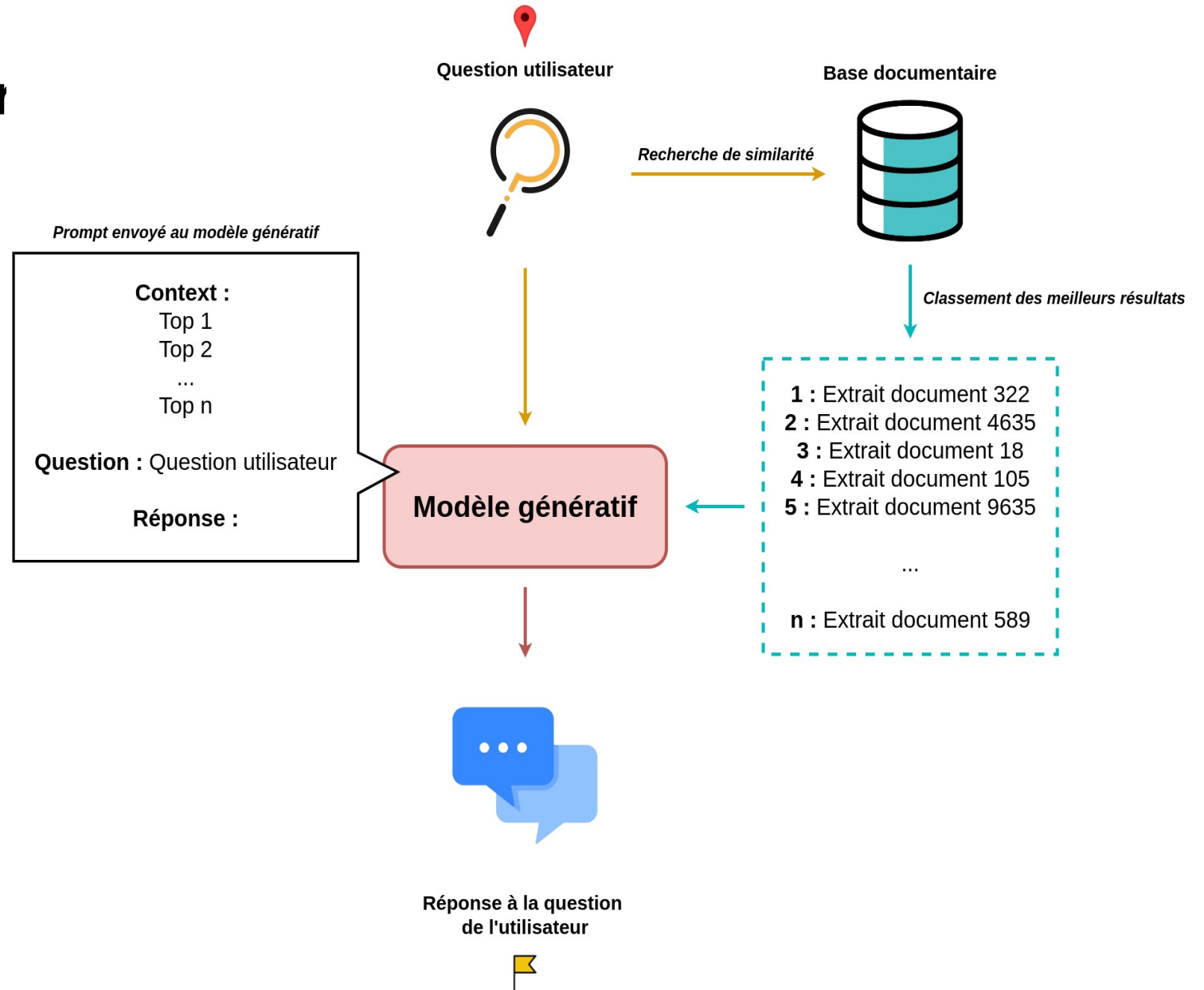
Pipe Function

plugin python apparaissant
comme un model

Filter

Traitement interceptant les
inputs ou les outputs

Pattern 1 — RAG pur



Pattern 1 — RAG pur

Quand

Question documentaire ciblée, besoin de preuve, réponse courte ou synthèse simple.

Mécanique

Le contexte récupéré est injecté dans le prompt via un template RAG.

Limite

Si trop de chunks sont injectés, coût, latence et dilution augmentent.

À surveiller

- chunking
- embedding
- reranking
- seuils de similarité
- re-index après changement embedding

Commencer par le RAG pur pour les cas simples, puis passer en tools-only quand la recherche doit devenir exploratoire.

Pattern 2 — RAG agentique, KB as tools



RAG classique

On injecte automatiquement les passages dans le prompt. Simple, mais coûteux si la récupération est trop large.

RAG agentique

Le modèle appelle des outils de recherche/lecture quand il en a besoin. Plus sobre, mais exige un bon prompt système et des tools fiables et bien documentés. Le LLM utilise des boucles de raisonnement type ReAct

Pourquoi c'est rationnel

Le modèle ne reçoit pas tout le corpus par défaut. Il appelle la recherche, puis lit seulement les passages nécessaires. Cela garde le préfixe plus stable et limite les tokens consommés.

Pattern 3 — Custom tools OWUI, sortir le déterministe du prompt

Squelette minimal

```
class Tools:
    class Valves(BaseModel):
        api_url: str

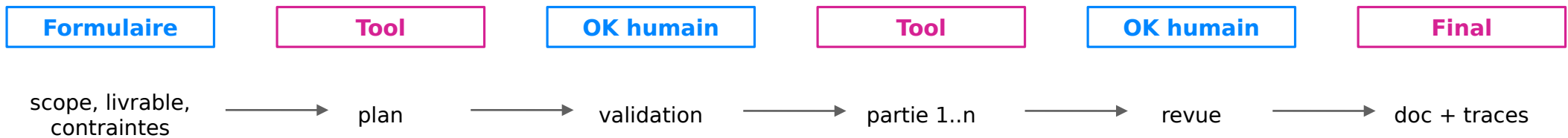
    async def analyser(self, document_id: str) ->
str:
    """Analyse le document et retourne les
écartés."""
    ...
```

- Un fichier Python
- Docstring courte et explicite: elle guide le modèle
- Type hints: nécessaires pour générer un schéma fiable
- Valves / UserValves = Arguments injectés: user, messages, fichiers, OAuth, event emitter

Un tool mal nommé/documenté ou trop générique est presque équivalent à un prompt mal écrit: le modèle hésite, choisit mal, ou n'appelle pas.

Ne pas faire “un agent magique”. Faire plusieurs tools petits: rechercher, lire, comparer, générer un plan, valider une checklist, produire un livrable.

Pattern 3 étendu: Tandem Tools + Prompt Templates



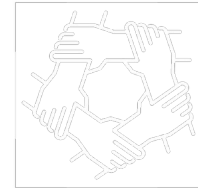
- Le prompt template contraint la demande et sélectionne les bonnes options
- Le tool porte la logique répétable, les accès et les contrôles
- Le workflow multi-étape reste compréhensible sans coder un orchestrateur complet
- L'humain valide les jalons qui changent le livrable

Exemple :

“Génère un plan de synthèse, attends validation, rédige partie par partie, puis fournis les points d’incertitude.”

Besoin de validation & mesure de Perf: datasets & PromptFoo

- **PromptFoo:** validation depuis datasets avec strategies & juges multiples (peut aussi faire du red teaming sécu)
- Création d'un dataset avec le métier (csv, excel, json) qui peut accepter des réponses approximatives, des critères multiples d'acceptation
- Avoir un juge fixe pour la validation et rendre comparable les modèles LLM
- Une fois le yaml configurer, c'est du bonheur
- En plus de la mesure/validation initiale le yaml/test PromptFoo peut être réutilisé pour non regression, voir de **l'optimisation**



Failure modes et tips

Le modèle n'appelle pas le tool

forcer le comportement via prompt système, documentation des tools, tests et exemples

Le tool renvoie trop de texte

résumer côté tool, pagination, limite stricte

La KB est mal indexée

chunking, embedding, rerank, re-index

L'agent a trop de droits

scopes, secrets, confirmation, audit

Native Tool Calling

En mode natif, la KB n'est pas forcément injectée automatiquement: le modèle doit appeler les tools de connaissance.

Sécurité tools

Un tool OWUI peut exécuter du Python arbitraire. Restreindre permissions, secrets, réseau et scopes.

Cache & contexte

Garder un préfixe stable; éviter d'injecter systématiquement des chunks ou citations qui changent à chaque tour.

RAG tuning

Embedding homogène, re-index si modèle d'embedding change, hybride BM25 + vector + rerank selon cas.

Événements UI

Certains event emitters sont moins adaptés au mode natif: tester statuts, citations et confirmations.

Évaluation

Mesurer qualité, coût, latence, erreurs tool, non-appels tool, satisfaction métier.

Types de tools utiles en entreprise

Synthèse grande taille

Plan/choix + lots/itératifs + fusion/formatage

Outils de simulation

Conception simulation + utilisation simu externe

Analyse conformité

checklist, écarts, incohérences

Génération documentaire

template, sections, style

Doc collaborative (multi-experts et agents IA)

Personnes + IAG + versions

Intégrations apps

GED, API, messagerie, traduction

Critère de priorisation

Prioriser les tools qui suppriment le plus de contexte répété ou qui fiabilisent une étape métier risquée.

Briques open source intégrables

RAGFlow

RAG/agents, parsing riche, API, MCP

OpenCode

agent de code terminal / IDE / desktop

OpenClaw

agent personnel local, messageries, skills

LLM-Wiki

connaissance structurée comme wiki outillé

Atomic Strata

profil/templates LLM-Wiki: à qualifier

Hermes Agent

mention possible, source à verrouiller

Message: OWUI n'a pas vocation à tout remplacer. Son intérêt est aussi de fournir un point d'entrée qui peut appeler ou encapsuler d'autres briques.

Du serveur proto à la Plateforme IA en cubes

Instances Open WebUI

Open WebUI / autres UI
tools, fonctions, workflows

Connaissance

KB, index, graph/wiki,
RAGFlow

Middleware

GED, API, apps métier

Modèles

gateway, quotas, routing

Gouvernance

RBAC, logs, eval, sécurité

Optimisation

- Tools, system prompt & skills optimisés avec DSPy / OpenTrace / GEPA OptimizeAnything avec un feedback sur des datasets. Possibilité de multi-objectif: réponse juste, tokens/turns.
- NEXT: finetuning de petits modèles SLM depuis traces de gros modèles

À retenir

- Partir de l'observation de workflows métier
- Découpler connaissance, prompts et outils
- Déplacer le déterministe dans les tools et un guidage "templatisé" avec HITL (human-in-the-loop)
- Garder le modèle sur le raisonnement et la synthèse
- Standardiser les entrées avant de multiplier les modèles
- Mesurer coût, latence, qualité => PromptFoo pour valider et évoluer sans régression

Questions